

TÍTULO DO PROJETO: Restaurantes

Autores:

- Rafael Brandão
- Felipe Dias Mac Dowell
- Rodrigo do Carmo Cruz
- Francisco Ferreira
- Ana Melissa Lima do Nascimento

Data: dd/mm/aaaa

1. Introdução

Este documento descreve as atividades desenvolvidas no projeto **Sistema de Gestão de Restaurantes**, elaborado pela equipe responsável pelo Tech Challenge da Pós-Tech em Arquitetura e Desenvolvimento Java. O projeto busca solucionar a necessidade de gerenciamento de usuários em um sistema de restaurantes, por meio do desenvolvimento de uma API REST que centraliza o cadastro, autenticação e administração de clientes e proprietários de restaurantes.

A solução foi desenvolvida utilizando **Java 21**, **Spring Boot**, **PostgreSQL** e **Docker Compose**, aplicando boas práticas de arquitetura, segurança e organização do código, com foco na criação de uma aplicação escalável, segura e de fácil manutenção.

2. Definição do Problema

Restaurantes que utilizam soluções digitais precisam de um sistema confiável para gerenciar o cadastro e a autenticação de usuários, garantindo que clientes e proprietários tenham acesso seguro às funcionalidades disponíveis. A ausência de um processo estruturado pode resultar em falhas de segurança, dificuldade na administração de usuários e baixa confiabilidade no acesso ao sistema.

Diante desse cenário, o projeto propõe o desenvolvimento de uma API REST responsável pelo gerenciamento de usuários, contemplando cadastro, autenticação e controle de acesso por meio de perfis de usuário. A solução busca oferecer uma base segura, organizada e escalável para futuras funcionalidades do sistema de gestão de restaurantes.

3. Levantamento de Requisitos

3.1 Event Storming

O levantamento de requisitos foi realizado por meio da identificação dos principais eventos de negócio relacionados ao gerenciamento de usuários do sistema. A partir dos fluxos da aplicação, foram mapeados os comandos executados pelos usuários, os eventos gerados e os agregados responsáveis por manter a consistência das informações.

O processo permitiu compreender o ciclo de vida do usuário dentro do sistema, desde o cadastro e autenticação até a atualização de dados, alteração de senha e exclusão da conta, além das regras de autorização por perfil e posse do recurso.

Resumo dos elementos

Comandos

- Cadastrar usuário;
- Realizar login;
- Buscar usuário;
- Atualizar dados do usuário;
- Alterar senha;
- Excluir usuário.

Eventos

- Usuário cadastrado;
- Usuário autenticado;
- Usuário atualizado;
- Senha alterada;
- Usuário removido;
- Acesso autorizado ou negado conforme perfil e permissões.
- **Agregados**
 - Usuário;
 - Endereço;
 - Perfil de acesso (Role).

Fluxo principal

- O usuário realiza o cadastro no sistema;
- Efetua a autenticação para obtenção do token JWT;
- Utiliza o token para acessar as funcionalidades protegidas;
- Gerencia seus próprios dados, incluindo atualização de informações e alteração de senha;
- Administradores possuem permissão para acessar e gerenciar qualquer usuário do sistema.

3.2 Mapeamento de Demandas e Necessidades

O sistema foi desenvolvido para atender à necessidade de um gerenciamento seguro e eficiente de usuários, permitindo o cadastro, autenticação e administração das informações de forma centralizada.

Perfis de usuários e necessidades

O sistema possui três perfis de acesso principais, definidos por meio da entidade Role e persistidos no banco de dados:

Cliente (ROLE_CUSTOMER)

Representa o usuário que utiliza o sistema como cliente. Pode realizar seu cadastro, autenticar-se, consultar seus dados, atualizar suas informações, alterar sua senha e excluir sua própria conta.

Proprietário de Restaurante (ROLE_OWNER)

Representa o proprietário de um restaurante. O perfil está previsto na modelagem de autorização da aplicação e é utilizado para representar o usuário responsável pelo restaurante. Na Fase 01, o escopo implementado está concentrado no gerenciamento de usuários, autenticação e autorização, não contemplando ainda funcionalidades específicas de gerenciamento de restaurantes.

Administrador (ROLE_ADMIN)

Representa o usuário responsável pela administração do sistema. Possui permissões superiores às dos demais perfis e pode consultar, atualizar e excluir usuários diferentes do usuário autenticado, conforme as regras de autorização implementadas.

O cadastro público não permite que um usuário atribua a si próprio o papel ROLE_ADMIN. O papel administrativo é disponibilizado por meio do seed administrativo ou de um fluxo autenticado autorizado.

3.3 Identificação dos Agregados (Domínios)

O modelo de domínio da aplicação possui como principal agregado a entidade User, que representa a identidade do usuário e concentra os principais relacionamentos utilizados pelas regras de negócio.

Agregado User

O User representa a raiz do agregado e contém os dados de identificação, credenciais, papéis de acesso e endereços relacionados ao usuário.

Entidade Address

A entidade Address representa os endereços associados a um usuário. Existe uma relação de um-para-muitos entre usuário e endereço, sendo o endereço dependente do usuário.

Entidade Role

A entidade Role representa os papéis de acesso utilizados para autorização. Os papéis são compartilhados entre usuários e relacionados por meio da tabela associativa user_roles, caracterizando uma relação muitos-para-muitos.

Entidade PasswordResetToken

A entidade PasswordResetToken representa os tokens utilizados no processo de recuperação de senha. Cada token pertence a um usuário, possui prazo de expiração e pode ser utilizado uma única vez.

Relações principais

- User → Address: 1;
- User → Role: N;
- User → PasswordResetToken: 1.

O User permanece como o núcleo do modelo, enquanto Address, Role e PasswordResetToken apoiam funcionalidades específicas relacionadas ao ciclo de vida e segurança do usuário.

4. Arquitetura do Sistema

4.1 Abordagem Domain-Driven Design (DDD)

O projeto utiliza conceitos de Domain-Driven Design (DDD) para organizar as entidades, responsabilidades e regras relacionadas ao domínio de gerenciamento de usuários. O agregado User representa o núcleo do modelo, relacionando-se com Address para representar os endereços, com Role para representar os perfis de acesso e com PasswordResetToken para suportar o processo de recuperação de senha.

A implementação utiliza uma arquitetura em camadas, separando controller, service, repository e entity. Os Controllers são responsáveis pela exposição da API REST, os Services concentram as regras de negócio, os Repositories abstraem o acesso ao PostgreSQL e as Entities representam o modelo de domínio. Componentes transversais, como segurança JWT, tratamento de exceções, mapeamento e configuração, complementam a estrutura da aplicação.

4.2 Definição dos Domínios

O sistema possui como domínio principal o **Gerenciamento de Usuários**, responsável pelas funcionalidades relacionadas ao cadastro, autenticação, consulta, atualização, alteração de senha e exclusão de usuários. O domínio também contempla as regras de autorização e o relacionamento dos usuários com seus papéis de acesso e endereços.

A partir da implementação existente, podem ser identificados os seguintes subdomínios e respectivos limites contextuais:

1. Domínio de Usuários

É o domínio central da aplicação e representa a identidade dos usuários do sistema. É responsável pelo cadastro, consulta, atualização, alteração de senha e exclusão de usuários.

Também concentra regras de negócio como:

- validação e unicidade de e-mail e login;
- normalização dos dados de entrada;
- armazenamento seguro da senha utilizando BCrypt;
- associação dos usuários aos seus papéis de acesso;
- associação dos usuários aos seus endereços;
- controle de acesso aos recursos;
- registro das informações de auditoria.

O agregado User representa a raiz do domínio e mantém as relações com Role e Address.

2. Domínio de Autenticação e Segurança

Responsável pela autenticação dos usuários e pelo controle de acesso às funcionalidades protegidas da API.

O domínio utiliza Spring Security e JWT. O usuário realiza o login por meio de suas credenciais e, após a validação, recebe um token JWT que deve ser enviado nas requisições protegidas.

Também fazem parte desse contexto as funcionalidades de recuperação de senha, utilizando tokens de uso único e com validade limitada.

O controle de autorização utiliza os papéis associados ao usuário e também verifica a posse do recurso. Dessa forma, operações realizadas sobre um usuário específico podem ser executadas pelo próprio usuário ou por um administrador.

3. Domínio de Perfis de Acesso

Responsável pela definição dos papéis utilizados para autorização na aplicação.

Os papéis existentes na implementação são:

- ROLE_CUSTOMER: representa o cliente;
- ROLE_OWNER: representa o proprietário/dono de restaurante;
- ROLE_ADMIN: representa o administrador do sistema.

Os papéis são persistidos na tabela roles e relacionados aos usuários por meio da tabela associativa user_roles, permitindo que um usuário possua mais de um papel.

O `ROLE_ADMIN` possui privilégios administrativos e pode acessar recursos de outros usuários. Já os usuários comuns possuem acesso aos próprios recursos conforme as regras de autorização da aplicação.

4. Domínio de Endereços

Responsável pelo gerenciamento dos endereços associados aos usuários.

Um usuário pode possuir um ou mais endereços, estabelecendo uma relação de 1 entre User e Address. O endereço depende do usuário ao qual está vinculado e é removido quando o usuário é excluído.

O domínio contempla informações como rua, número, complemento, bairro, cidade, estado e CEP.

5. Domínio de Recuperação de Senha

Responsável pelo processo de recuperação de senha quando o usuário não possui ou não lembra sua senha atual.

O fluxo é dividido em duas operações:

- solicitação de recuperação de senha por e-mail;
- redefinição da senha utilizando um token temporário.

Os tokens são armazenados de forma segura, utilizando hash SHA-256, possuem prazo de expiração e podem ser utilizados somente uma vez.

Limites Contextuais

O **Gerenciamento de Usuários** representa o núcleo do sistema. Os demais domínios atuam como contextos de apoio às funcionalidades relacionadas ao usuário.

A relação pode ser representada da seguinte forma:

Usuário → Perfis de Acesso

Define os papéis e permissões associados ao usuário.

Usuário → Endereços

Define os endereços pertencentes ao usuário.

Usuário → Autenticação e Segurança

Permite autenticar o usuário e controlar o acesso aos recursos protegidos.

Usuário → Recuperação de Senha

Permite recuperar e redefinir a senha por meio de tokens temporários.

Interação entre os Domínios

Durante o cadastro, o domínio de Usuários recebe os dados do novo usuário, valida as informações, associa os papéis permitidos e registra os endereços informados.

Durante o login, o domínio de Autenticação valida as credenciais do usuário e gera um token JWT. Esse token passa a ser utilizado nas requisições seguintes para identificar o usuário autenticado e aplicar as regras de autorização.

Nas operações de consulta, atualização, alteração de senha e exclusão, o sistema verifica se o usuário autenticado possui permissão para acessar o recurso. Usuários comuns podem operar seus próprios dados, enquanto o `ROLE_ADMIN` pode administrar qualquer usuário.

O domínio de Recuperação de Senha interage com o domínio de Usuários para localizar o usuário e permitir a alteração segura de sua senha por meio de um token temporário.

Resumo da Estrutura

A arquitetura da aplicação utiliza o padrão de **Arquitetura em Camadas**, separando as responsabilidades em `controller`, `service`, `repository` e `entity`. Os `controllers` expõem a API REST, os `services` concentram as regras de negócio, os `repositories` realizam o acesso aos dados e as entidades representam o modelo de domínio.

A aplicação utiliza ainda componentes transversais para segurança, tratamento de erros, configuração, mapeamento e objetos de valor. O fluxo principal segue a sequência:

Cliente HTTP → Controller → Service → Repository → PostgreSQL

A autenticação JWT e o mecanismo de autorização atuam de forma transversal, protegendo os endpoints que exigem autenticação.

Essa organização permite manter as regras de negócio concentradas na camada de serviço, reduzir o acoplamento entre as camadas e facilitar a manutenção e evolução da aplicação.

4.3 Modelagem Conceitual

O sistema é composto por três domínios principais, responsáveis pelo gerenciamento de usuários, perfis de acesso e endereços.

Domínios identificados

- **Usuários:** responsável pelo cadastro, autenticação, atualização, consulta e exclusão de usuários.
- **Perfis de Acesso:** gerencia os papéis (roles) utilizados para controlar as permissões de acesso à aplicação.
- **Endereços:** responsável pelo armazenamento dos endereços associados aos usuários.

Interação entre os domínios

O domínio **Usuários** é o núcleo da aplicação e interage diretamente com os domínios de **Perfis de Acesso** e **Endereços**. Durante o cadastro e a manutenção dos usuários, são atribuídos perfis de acesso e vinculados um ou mais endereços ao usuário.

Resumo da arquitetura

A arquitetura é organizada em camadas, separando `controllers`, `services`, `repositories` e entidades. O domínio de Usuários concentra as principais regras de negócio, enquanto os domínios de Endereços e Perfis de Acesso fornecem suporte às funcionalidades da aplicação, mantendo baixo acoplamento e facilitando a manutenção do sistema.

5. Documentação do Projeto

- [Apresente a documentação completa do projeto, incluindo o Event Storming, a descrição da arquitetura proposta e os resumos dos modelos conceituais.]

- [Assegure que a documentação esteja clara, organizada e abrangente, permitindo a compreensão completa do projeto.]

6. Implementação

O sistema foi desenvolvido utilizando **Java 21** e **Spring Boot**, seguindo uma arquitetura em camadas composta por controllers, services, repositories e entidades. Como apoio ao desenvolvimento, foram utilizados PostgreSQL para persistência dos dados, Spring Security para autenticação e autorização com JWT, Flyway para versionamento do banco de dados e Docker Compose para execução da aplicação.

A implementação adota práticas inspiradas em Domain-Driven Design (DDD), organizando as entidades de domínio, separando as responsabilidades entre as camadas e concentrando as regras de negócio na camada de serviço.

Os requisitos levantados foram implementados por meio das funcionalidades de cadastro de usuários, autenticação, gerenciamento de perfis de acesso, consulta, atualização, alteração de senha e exclusão de usuários, garantindo validações de negócio e controle de acesso conforme o perfil do usuário.

Fluxo de Cadastro e Autenticação

- O usuário realiza o cadastro informando seus dados pessoais, endereço e perfil permitido.
- O sistema valida as informações e persiste os dados no banco de dados.
- Após o cadastro, o usuário realiza o login utilizando seu login e senha.
- Em caso de sucesso, é gerado um token JWT, utilizado para acessar os demais endpoints protegidos da aplicação.

Fluxo de Gerenciamento de Usuários

- O usuário autenticado utiliza o token JWT para acessar as funcionalidades da API.
- O sistema valida as permissões do usuário antes de executar cada operação.
- O usuário pode consultar, atualizar seus dados, alterar sua senha ou excluir sua conta, enquanto administradores possuem acesso às operações administrativas sobre qualquer usuário

6.3 Tipos de Usuários e Rotas Disponíveis

A API possui controle de acesso baseado em papéis (Role). Os principais tipos de usuários são ROLE_CUSTOMER, ROLE_OWNER e ROLE_ADMIN.

As rotas da API são versionadas por meio do prefixo /api/v1.

Autenticação

Método	Rota	Descrição	Acesso
POST	/api/v1/auth/login	Realiza autenticação e retorna o token JWT	Público

POST	/api/v1/auth/forgot-password	Solicita recuperação de senha	Público
POST	/api/v1/auth/reset-password	Redefine a senha utilizando token de recuperação	Público

Usuários

Método	Rota	Descrição	Acesso
POST	/api/v1/users	Realiza o cadastro de um novo usuário	Público
GET	/api/v1/users	Lista usuários de forma paginada	Autenticado
GET	/api/v1/users?name={nome}	Busca usuários por nome	Autenticado
GET	/api/v1/users/{id}	Consulta um usuário pelo UUID	Próprio usuário ou ROLE_ADMIN
PUT	/api/v1/users/{id}	Atualiza os dados do usuário	Próprio usuário ou ROLE_ADMIN
PATCH	/api/v1/users/{id}/password	Altera a senha do usuário	Próprio usuário ou ROLE_ADMIN
DELETE	/api/v1/users/{id}	Exclui um usuário	Próprio usuário ou ROLE_ADMIN

O cadastro é público, porém não permite que o usuário atribua a si próprio o papel ROLE_ADMIN.

As operações direcionadas a um usuário específico utilizam autorização por posse do recurso. Dessa forma, um usuário comum somente consegue acessar ou alterar seus próprios dados, enquanto um administrador pode realizar operações sobre qualquer usuário.

As consultas de coleção, como a listagem e a busca por nome, exigem apenas autenticação.

Documentação da API

A aplicação disponibiliza documentação interativa por meio do Swagger/OpenAPI:

- Swagger UI: `/swagger-ui.html`
- OpenAPI JSON: `/v3/api-docs`

O Swagger permite autenticar utilizando o token JWT obtido no endpoint de login e executar os endpoints protegidos diretamente pela interface.

Monitoramento

A aplicação também disponibiliza endpoints de monitoramento por meio do Spring Boot Actuator:

- `/actuator/health`
- `/actuator/info`
- `/actuator/metrics`

Os endpoints de monitoramento possuem diferentes níveis de proteção conforme a configuração de segurança da aplicação.

7. Considerações Finais

O levantamento de requisitos por meio do Event Storming contribuiu para identificar os principais processos do sistema, permitindo compreender o fluxo das funcionalidades e organizar os elementos do domínio. A utilização de conceitos inspirados em Domain-Driven Design (DDD) auxiliou na definição das responsabilidades das entidades e na organização da arquitetura da aplicação.

Durante o desenvolvimento, os principais desafios estiveram relacionados à implementação da autenticação com JWT, ao controle de acesso por perfis de usuários, à validação das regras de negócio e à definição de uma arquitetura organizada e de fácil manutenção. Esses desafios foram superados com a adoção de boas práticas de desenvolvimento, arquitetura em camadas, versionamento do banco de dados com Flyway e utilização dos recursos oferecidos pelo ecossistema Spring.

Como resultado, foi desenvolvida uma API REST segura, organizada e escalável para o gerenciamento de usuários de um sistema de restaurantes. A solução proporciona maior controle sobre o acesso à aplicação, centraliza o gerenciamento dos usuários e estabelece uma base sólida para a implementação de novas funcionalidades nas próximas etapas do projeto.

8. Artefatos

Artefatos

Os seguintes artefatos compõem a entrega do projeto:

Artefato	Descrição	Localização
Código-fonte	Implementação completa da API REST desenvolvida em Java com Spring Boot.	https://github.com/felipe11dias/postech-restaurantes-app
Código da aplicação	Diretório contendo a implementação da aplicação, incluindo controllers, services, repositories, entidades e demais componentes.	https://github.com/felipe11dias/postech-restaurantes-app/tree/main/fase-01/camadas/restaurantes
Relatório Técnico	Documento contendo a descrição do projeto, arquitetura, modelagem, implementação e decisões técnicas.	Diretório fase-01/relatorio do repositório GitHub.
Documentação da API	Documentação interativa dos endpoints utilizando Swagger/OpenAPI.	Disponível na aplicação em /swagger-ui/index.html.
Coleção Postman	Coleção para testes dos endpoints da API.	Diretório fase-01/camadas/restaurantes/postman do repositório GitHub.
Scripts de Banco de Dados	Scripts de criação e versionamento do banco utilizando Flyway.	Diretório src/main/resources/db/migration do projeto.
Docker	Arquivos para execução da aplicação e do banco de dados via Docker Compose.	Diretório raiz da aplicação (Dockerfile e docker-compose.yml).

Prints da Api	Imagens das respostas, cabeçalho, status dos endpoints do sistema.	Diretório dos prints Prints dos endpoints
---------------	--	--

9. Referências

- EVANS, Eric. *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Boston: Addison-Wesley, 2003.
- FOWLER, Martin. *Patterns of Enterprise Application Architecture*. Boston: Addison-Wesley, 2002.
- Spring. **Spring Boot Documentation**. Disponível em: <https://docs.spring.io/spring-boot/index.html>. Acesso em: 27 jul. 2026.
- Spring. **Spring Security Reference**. Disponível em: <https://docs.spring.io/spring-security/reference/>. Acesso em: 27 jul. 2026.
- PostgreSQL Global Development Group. **PostgreSQL Documentation**. Disponível em: <https://www.postgresql.org/docs/>. Acesso em: 27 jul. 2026.
- Flyway. **Flyway Documentation**. Disponível em: <https://documentation.red-gate.com/flyway>. Acesso em: 27 jul. 2026.
- Docker Inc. **Docker Documentation**. Disponível em: <https://docs.docker.com/>. Acesso em: 27 jul. 2026.
- OpenAPI Initiative. **OpenAPI Specification**. Disponível em: <https://swagger.io/specification/>. Acesso em: 27 jul. 2026.
- GitHub. **Repositório do Projeto – Tech Challenge – Sistema de Gestão de Restaurantes**. Disponível em: <https://github.com/felipe11dias/postech-restaurantes-app>. Acesso em: 27 jul. 2026.

